OptiLine-Py

Raceline Optimisation Toolkit

Package Manual — Version 0.2.8

Amir Ali Farzin

amirali.farzin@anu.edu.au

Abstract

OptiLine-Py is a Python package for computing optimal racing lines on closed tracks. It provides routines for minimum-curvature and minimum lap-time trajectory optimisation, shortest-path and optimal-shortest-path planning, cubic and clothoid spline geometry, and full kinematic profile generation (velocity, acceleration, and lap time). Two interchangeable velocity-profile engines are provided: the built-in forward–backward solver and, optionally, the semi-analytical FBGA (*Fb2d*) solver, which can be selected throughout the package via a single `vel_solver` switch. The package combines quadratic programming, evolutionary strategies (CMA-ES), zero-order Gaussian random methods (ZORM), direct collocation via CasADi/IPOPT, and a blackbox direct-alpha optimiser into a unified, extensible interface aimed at autonomous racing research. The package includes a procedural map generator (`MapBuilder`), enabling the creation of arbitrarily large synthetic track datasets for machine-learning research.

Contents

1 Introduction **3**

2 Installation **3**
- 2.1 Requirements . . . 3
- 2.2 Installing with pip . . . 4
- 2.3 Installing from source (Poetry) . . . 4
- 2.4 Installing using a wheel file . . . 4

3 Package Structure **4**

4 Module: `utils.py` **5**
- 4.1 `calc_splines` . . . 5
- 4.2 `interp_splines` . . . 6
- 4.3 `calc_spline_lengths` . . . 6
- 4.4 `create_raceline` . . . 6
- 4.5 `calc_head_curv_an` . . . 7
- 4.6 `spline_extraction_ops` . . . 7
- 4.7 `interp_corridor_rows` – Interior-Point Corridor Constraints . . . 7
- 4.8 `H_f` – Minimum-Curvature QP Matrices . . . 8
- 4.9 `import_veh_dyn_info` . . . 8
- 4.10 `reftrack_to_mlts_track` . . . 9
- 4.11 `conv_filt` . . . 9
- 4.12 `load_ggv_envelope` – Shared G-G-V Envelope Reconstruction . . . 9
- 4.13 Track sampling and reconstruction helpers . . . 10
- 4.14 Centreline projection toolkit . . . 11

5 Module: `solvers.py` **11**
- 5.1 `opt_min_curv` – Minimum-Curvature QP . . . 12
- 5.2 `OSP`, `ShortestPath` – Optimal Shortest Path QP Matrices and Feasible Path . . . 12
- 5.3 Class `Opt_min_CurvTime` . . . 14
- 5.4 Class `ConstrainedCMAES_t` . . . 16
- 5.5 Class `ZORM` . . . 17
- 5.6 Class `Blackbox_raceline` . . . 17
- 5.7 Class `PointMassOCP` – Point-Mass Minimum-Lap-Time OCP . . . 21
- 5.8 Class `Clothoid_raceline` . . . 22

6 Module: `KinematicProfs.py` **22**
- 6.1 `calc_vel_profile` . . . 23
- 6.2 `calc_vel_profile_fb2d` . . . 24
- 6.3 `calc_vel_profile_solver` . . . 25
- 6.4 `calc_ax_profile` . . . 26
- 6.5 `calc_t_profile` . . . 26
- 6.6 `curvature_profile` and `curvature_profile2` . . . 26
- 6.7 `cumulative_distances` . . . 26

7 Module: `opt_mintime.py` **26**

8 Module: `map_builder.py` **27**
- 8.1 Track generation pipeline . . . 27
- 8.2 Track archetypes . . . 27
- 8.3 Class `MapBuilder` . . . 28

8.4 Functional API 29
8.5 Command-line usage 29

9 Usage Examples **29**
9.1 `General_test.py` – Geometric, Proxy, and Blackbox Optimisation 29
9.2 `General_test_optmintime.py` – Minimum Lap-Time Optimal Control 34
9.3 `map_building_test.py` – MapBuilder Demonstrations 35
9.4 `Map_builder_batch_gen.py` – Command-Line Batch Generation 36

10 GGV Diagram Format **36**

11 Track Map Format **37**

12 Dependencies and Licences **37**

1 Introduction

Racing-line optimisation is the problem of choosing, for every point around a closed circuit, a lateral position within the drivable corridor such that a chosen cost (lap time, path curvature, path length, ...) is minimised subject to track-boundary and vehicle-dynamics constraints.

OptiLine-Py addresses this problem at multiple levels of fidelity:

1. **Geometric methods** – minimum-curvature (QP) and optimal-shortest-path (QP) formulations that operate purely on track geometry, producing a smooth spline raceline without requiring a vehicle model.
2. **Lap-time proxy methods** – the combined curvature-and-time optimiser `Opt_min_CurvTime` uses the kinematic velocity profile as a fast surrogate for lap time and drives the search with CMA-ES or ZORM over the spline segment-length vector.
3. **Blackbox direct-alpha optimisation** – the `Blackbox_raceline` class optimises the lateral shift vector $\boldsymbol{\alpha}$ directly, treating the kinematic lap-time oracle (or any user-supplied callable) as a blackbox cost function and using ZORM or CMA-ES for the outer search.
4. **Optimal-control methods** – a full single-track minimum-time optimal-control problem solved by direct Gauss–Legendre collocation via CasADi and IPOPT (`opt_mintime`).

The package is structured as follows:

Module	Role
`solvers.py`	Core optimisation solvers and raceline classes
`utils.py`	Spline utilities, track preprocessing, QP matrix builders
`map_builder.py`	Procedural race-track and occupancy-grid map generator
`opt_mintime.py`	Minimum lap-time optimal control (CasADi/IPOPT)
`KinematicProfs.py`	Velocity, acceleration, and lap-time profile computation

Parts of this codebase are adopted from https://github.com/TUMFTM/global_racetrajectory_optimization.

2 Installation

2.1 Requirements

OptiLine-Py requires Python 3.10–3.12 and the following libraries:

Library	Version	Purpose
numpy	$\geq 1.24, < 3$	Array operations
scipy	$\geq 1.10, < 2$	Interpolation, linear algebra
matplotlib	$\geq 3.7, < 4$	Plotting
pillow	$\geq 10.0, < 12$	PNG occupancy-grid map rendering
quadprog	$\geq 0.1.13, < 0.2$	Quadratic programming
casadi	$\geq 3.7, < 4$	Automatic differentiation / IPOPT
fbga-py	$\geq 0.1.0$	Semi-analytical FBGA velocity solver (`calc_vel_profile_fb2d`)
MLTS-point-mass	$\geq 0.1.0$	Point-mass minimum-lap-time OCP (`PointMassOCP`)
numba	≥ 0.59	JIT compilation of the velocity-profile kernels

fbga-py provides Python bindings to the FBGA (*Forward–Backward Generic Acceleration constraints*) library (https://github.com/DRIVEWISE/FBGA) and is required only for the `vel_solver='fb2d'` velocity-profile option (Section 6). It is imported lazily, so all other functionality works even if it is absent.

numba, added as a dependency in v0.2.7, JIT-compiles the inner kernels of `calc_vel_profile` (Section 6.1). It is not required for correctness: if it is absent, or installed but broken, the import is caught and a transparent pure-Python fallback is used instead. It is declared as a dependency because the velocity profile is evaluated in the inner loop of every zeroth-order solver, so whether it is present changes the reported runtimes — about 2× on a 400-point profile — and benchmarks should not depend on how an environment happens to be assembled. Note that *numba* tracks *numpy* closely and in practice caps it below the < 3 this package allows.

MLTS-point-mass, added as a dependency in v0.2.6, provides the external point-mass minimum-lap-time OCP behind `PointMassOCP` (Section 5.7). Like *fbga-py* it is imported lazily, only when the solver is actually constructed, so a partially installed environment still runs everything else.

A note on virtual environments. Both lazy imports fail in the same way if the tests or notebooks are launched from a different interpreter than the one OptiLine was installed into — most commonly the system Python instead of the project virtualenv, which by default does not see the user site-packages. The raised `ImportError` names `sys.executable` for that reason. From the source tree, use

```
poetry install
poetry run python tests/General_test.py
```

2.2 Installing with pip

```
pip install OptiLine-Py
```

2.3 Installing from source (Poetry)

```
git clone https://github.com/amirali78frz/OptiLine.git
cd OptiLine
poetry install
```

2.4 Installing using a wheel file

```
pip install optiline_py-0.2.7-py3-none-any.whl
```

3 Package Structure

```
OptiLine/
|-- src/
|   |-- OptiLine/
|       |-- __init__.py
|       |-- solvers.py          # Optimisation algorithms
|       |-- utils.py            # Helper files
|       |-- map_builder.py      # Synthetic track and map generator
|       |-- opt_mintime.py      # Min-time optimal control
|       `-- kinematicProfs.py   # Velocity/acceleration/time profiles
|-- tests/
```

```
|   |-- General_test.py              # End-to-end geometric + proxy
    optimisation
|   |-- map_building_test.py         # MapBuilder demonstrations
|   |-- Map_builder_batch_gen.py     # CLI batch map generation script
|   `-- General_test_optmintime.py   # End-to-end min-time optimal
    control
|-- params/
|   `-- racecar.ini              # Vehicle dynamics parameters
|-- maps/                        # Track CSV files
|-- poetry.lock
`-- pyproject.toml
```

4 Module: utils.py

utils.py provides spline mathematics, track-preprocessing helpers, and QP matrix builders used by the solvers.

4.1 calc_splines

```
coeffs_x, coeffs_y, M, normvec_normalized = calc_splines(
    path,
    el_lengths=None,
    psi_s=None, psi_e=None,
    use_dist_scaling=True,
)
```

Solves for curvature-continuous (C2) cubic spline coefficients through a sequence of track points. The spline passes through every point and matches heading and curvature at each knot. For a closed track, periodic boundary conditions are enforced automatically.

Parameters

Parameter	Description
path	(n+1, 2) array of (x, y) points; last row repeats first for a closed track
el_lengths	(n,) segment lengths [m]; computed from Euclidean distance if None
psi_s, psi_e	Boundary headings (rad) for the unclosed case
use_dist_scaling	Scale the spline system by segment lengths for better conditioning

Returns

Return	Description
coeffs_x	(n,4) cubic coefficients for the x-coordinate
coeffs_y	(n,4) cubic coefficients for the y-coordinate
M	(n,n) spline system matrix (used by QP solvers)
normvec_normalized	(n,2) outward unit normal vectors at each control point

4.2 interp_splines

```
path_interp, inds_interp, t_interp, dists_interp = interp_splines(
    coeffs_x, coeffs_y, spline_lengths,
    incl_last_point=False,
    stepsize_approx=None,
    stepsize_fixed=None,
)
```

Interpolates a 2-D cubic spline at a uniform arc-length step size and returns sample points, spline-segment indices, local spline parameters ($t \in [0, 1]$), and cumulative distances. Exactly one of `stepsize_approx` or `stepsize_fixed` must be supplied.

4.3 calc_spline_lengths

```
spline_lengths = calc_spline_lengths(
    coeffs_x, coeffs_y,
    quickndirty=False,
    no_interp_points=15,
)
```

Numerically integrates the arc length of each cubic spline segment using Gaussian quadrature. Returns a `(n,)` array of segment lengths in metres.

4.4 create_raceline

```
(raceline, a_opt,
 coeffs_x_opt, coeffs_y_opt,
 spline_inds_opt, t_vals_opt,
 s_points_opt, spline_lengths_opt,
 el_lengths_opt) = create_raceline(
    refline, normvec_normalized, alpha, stepsize_interp)
```

Applies lateral offsets α_i along the unit normals to the reference line, fits new cubic splines through the resulting control points, and interpolates the raceline at the requested arc-length step size.

Parameters

Parameter	Description
`refline`	`(n,2)` reference line (x, y)
`normvec_normalized`	`(n,2)` unit normals (from **`calc_splines`**)
`alpha`	`(n,)` lateral shift vector [m]; positive shifts toward the right boundary
`stepsize_interp`	Target arc-length spacing of the output raceline [m]

Since v0.2.2 the function emits a `RuntimeWarning` when the reference track's node spacing is strongly non-uniform (longest/shortest spline segment > 5). A short segment forces its cubic to curl sharply in order to match its neighbours' tangents, which silently corrupts the curvature and hence the velocity profile. A coarse `stepsize_interp` may never sample the offending segment, so the defect can stay hidden until the step size is reduced; re-sample the reference track with (near-)uniform spacing instead.

4.5 calc_head_curv_an

```
psi, kappa = calc_head_curv_an(
    coeffs_x, coeffs_y,
    ind_spls, t_spls,
    calc_curv=True,
)
```

Analytically evaluates heading ψ and curvature κ from cubic spline coefficients at the given (segment index, local parameter) pairs.

4.6 spline_extraction_ops

```
A_inv, T_b, T_c = spline_extraction_ops(
    A,                  # spline system matrix from calc_splines
    no_points,          # number of reference-track points
    closed=True,
)
```

Returns the inverse of the spline system matrix together with the two coefficient-extraction products used by the minimum-curvature formulation,

$$T_b \;=\; A_{\mathrm{ex},b}\,A^{-1}, \qquad T_c \;=\; A_{\mathrm{ex},c}\,A^{-1},$$

where $A_{\mathrm{ex},b}$ picks the b_i coefficients (first derivative at each node) and $A_{\mathrm{ex},c}$ picks $2c_i$ (second derivative, hence curvature). This is the quantity **H_f**, **opt_min_curv** and **OSP** build their QP matrices from; it is exposed for callers writing their own QP on the same linearisation.

Request all three in one call when more than one is needed: forming A^{-1} dominates the cost, and the returned A^{-1} can also be handed to **interp_corridor_rows** via its A_inv argument.

4.7 interp_corridor_rows – Interior-Point Corridor Constraints

```
G_add, h_add = interp_corridor_rows(
    A,                  # spline system matrix from calc_splines
    reftrack,           # (n,4): x, y, w_right, w_left
    normvectors,        # (n,2): unit normals
    w_veh,              # full vehicle width [m] (clearance = w_veh/2
    per side)
    closed=True,
    n_interp_con=2,     # interior points constrained per spline
   segment
    A_inv=None,         # reuse an inverse the caller already has
)
```

A_inv lets a caller that has already inverted A—for instance via **spline_extraction_ops**—hand it over, rather than have it formed a second time; inverting A is the dominant cost of this function. Leave it at None and the inverse is computed internally.

The QPs of this package constrain $\boldsymbol{\alpha}$ only *at* the reference-track nodes. The raceline actually driven is the cubic spline *through* those nodes, and between two nodes that spline bulges outside the corridor—by up to 0.27 m on an 18 m node spacing. Reducing the node spacing removes this but scales the number of optimisation *variables*, which is expensive.

Instead, note that the spline coefficients solve $A\,\mathbf{c} = \mathbf{q} + M\boldsymbol{\alpha}$, so the position at any *fixed* (segment i, t) is affine in $\boldsymbol{\alpha}$. Measuring the deviation from the reference spline at the same (i, t)

cancels the constant part, leaving a purely *linear* map $\boldsymbol{\alpha} \mapsto n_i(t)$:

$$n_i(t) \;=\; [B_t\,\boldsymbol{\alpha}]_i, \qquad B_t \;=\; \mathrm{diag}(\hat{n}_t^x)\,E_t A^{-1} M_x + \mathrm{diag}(\hat{n}_t^y)\,E_t A^{-1} M_y,$$

where E_t evaluates $[1,\, t,\, t^2,\, t^3]$ on each segment and $\hat{n}_t$ is the node normal interpolated to (i,t). Bounding $n_i(t)$ at a few interior t values therefore adds constraint *rows* while leaving the number of variables unchanged. With `n_interp_con`$= k$ the constrained parameters are $t = 1/(k{+}1), \ldots, k/(k{+}1)$ and $2k\,n_{\text{splines}}$ rows are returned.

Setting `n_interp_con`$= 0$ returns empty arrays and restores the node-only behaviour of versions $\leq$ 0.2.2.

Accuracy and cost

Within the QP's own linear model the interior constraint is satisfied exactly. On the *final* interpolated raceline a small residual remains, because the QP freezes the spline parameterisation at the reference geometry while **`create_raceline`** re-fits the spline through the shifted points—the same linearisation gap that **`opt_min_curv`** reports as `curv_error_max`. On a 220-node circuit the measured interior overshoot falls from 0.133 m to 0.006 m. Because only rows are added, the cost is modest: on a 381-node circuit the minimum-curvature QP goes from 0.48 s to 0.52 s (+8%), against roughly 700 s if the same accuracy is pursued by refining the node spacing instead.

4.8 `H_f` – Minimum-Curvature QP Matrices

```
H, f, G, h = H_f(
    reftrack,            # (n,4): x, y, w_right, w_left
    normvectors,         # (n,2): unit normals
    A,                   # (n,n): spline system matrix from calc_splines
    kappa_bound,         # max curvature [1/m]
    w_veh,               # vehicle half-width [m] (safety clearance)
    closed=True,
    n_interp_con=2,      # interior corridor points per segment (0 = node-only)
)
```

Assembles the QP matrices (H, $\mathbf{f}$, G, $\mathbf{h}$) for the minimum-curvature problem. H encodes the spline-curvature Hessian; G and $\mathbf{h}$ encode the track-boundary box constraints on $\boldsymbol{\alpha}$, with per-side clearance `w_veh`:

$$-(w_{\text{left},i} - \delta) \;\leq\; \alpha_i \;\leq\; (w_{\text{right},i} - \delta), \qquad \delta = \texttt{w_veh}.$$

Since v0.2.3, `n_interp_con` additionally constrains interior points of each spline segment via **`interp_corridor_rows`** (Section 4.7), so that the *interpolated* raceline stays inside the corridor and not merely the nodes. Pass `n_interp_con`$= 0$ for the pre-0.2.3 node-only constraints.

`Opt_min_CurvTime` and **`Blackbox_raceline`** build their inner QP through **`H_f`**. Since v0.2.4 both expose `n_interp_con` as a constructor argument and forward it; before that they were pinned to the default, so a caller that chose a different value for **`opt_min_curv`** would silently get a *different* minimum-curvature solution from the class-based solvers.

The matrices are ready for *quadprog*: `solve_qp(H, -f, -G.T, -h, 0)`.

4.9 `import_veh_dyn_info`

```
ggv, ax_max_machines = import_veh_dyn_info(
    ggv_import_path, ax_max_machines_import_path)
```

Loads the GGV (velocity–grip–grip) diagram and the maximum machine traction table from CSV files. The GGV is an $(n_v \times 3)$ array with columns $[v,\ a_{x,\max},\ a_{y,\max}]$. The traction table is an $(m \times 2)$ array with columns $[v,\ a_{x,\text{drive}}]$.

4.10 `reftrack_to_mlts_track`

Converts an OptiLine reference track into the track table expected by the external *MLTS-point-mass* solver (Section 5.7).

```
track_df = utils.reftrack_to_mlts_track(reftrack)
```

The reference line is fitted with the same closed cubic splines used everywhere else in the package, so that the arc length, the heading and the curvature of the returned table are all evaluated on *one and the same* curve:

$$s_i = \int_0^{\zeta_i} \|\mathbf{r}'(\sigma)\|\,\mathrm{d}\sigma, \qquad \theta_i = \operatorname{atan2}(y_i', x_i'), \qquad \kappa_i = \frac{x_i' y_i'' - y_i' x_i''}{\left(x_i'^2 + y_i'^2\right)^{3/2}}. \tag{1}$$

This matters in practice: if a track table's abscissa belongs to a different curve than its tabulated coordinates, the curvilinear lap time reported by the OCP and the lap time implied by its own exported (x, y) path differ by exactly the ratio between the two length scales. Building the table from the splines removes that failure mode by construction, leaving only the chord-to-arc term $s \approx c\,(1 + (\kappa c)^2/24)$.

Conventions. OptiLine stores the heading as $\psi = \operatorname{atan2}(\mathrm{d}y, \mathrm{d}x) - \pi/2$ (so $\psi = 0$ is north) and its normal vectors point to the *right* of the direction of travel, whereas *MLTS-point-mass* expects `dir_mid_line` $= \operatorname{atan2}(\mathrm{d}y, \mathrm{d}x)$ and a lateral coordinate n that is positive to the *left*. Both are handled by the converter, so a lateral offset returned by the solver maps onto an OptiLine lateral shift as $\alpha = -n$. The heading is unwrapped so that the solver can spline it across the $\pm\pi$ branch cut.

Closing point. If the last row of `reftrack` duplicates the first it is dropped before the splines are fitted. Keeping it would leave a near-zero-length closing segment, which produces a spurious curvature spike at the seam — the same degeneracy discussed in Section 4.7.

pandas is imported inside the function (it ships with *MLTS-point-mass*), so it is not a dependency of OptiLine itself.

4.11 `conv_filt`

```
signal_out = conv_filt(signal, filt_window, closed)
```

Applies a symmetric moving-average (box-kernel) filter to a 1-D signal. `filt_window` is the number of taps (must be odd). `closed` controls whether wrap-around padding is used.

4.12 `load_ggv_envelope` – Shared G-G-V Envelope Reconstruction

```
env = load_ggv_envelope(
    source,              # path to the envelope JSON, or the already-loaded dict
    interp = 'mlts',     # 'mlts' (GGVConstr) or 'linear' (bilinear on the grid)
    n_v    = 200,        # velocity samples of the lookup grid ('mlts' only)
```

```
    n_ay    = 200,        # lateral-acceleration samples ('mlts' only)
    ggv_scales = None,    # optional grip scales passed to GGVConstr ('mlts' only)
)
# env: gg_upper, gg_lower, gg_range, g, v_max, vel_vec, ay_vec, ...
```

Builds the three g-g bound callables that **calc_vel_profile_fb2d** and the zeroth-order solvers expect, from a tabulated g-g-v envelope. Having one entry point for this matters for cross-method fairness: every solver in the package then rides the *same* reconstruction of the *same* file.

Two reconstructions are offered:

- `interp='mlts'` (default) builds the envelope with `mlts_point_mass.GGVConstr` and samples it onto a fast lookup grid. This is the construction the point-mass OCP is itself constrained with, so comparisons against that reference are like-for-like. The exact polar constraint $g(a_x, a_y, V) \leq 1$ is also returned, as `env['g']`, for exceedance auditing.
- `interp='linear'` interpolates bilinearly straight on the tabulated grid. Faithful to the file, but it reproduces the file's artefacts as well—an envelope fitted to close at top speed stores $a_{x,\min} = 0$ there, which removes most of the braking capability over the last few m/s.

`v_max` is always taken from the *untrimmed* velocity axis, so trimming collapsed rows cannot silently lower the speed cap.

Classes `Bilinear` and `Linear1D`

The returned callables are instances of these two module-level classes: **Bilinear** evaluates $f(a_y, V)$ on a uniform grid and **Linear1D** evaluates $f(V)$, both with linear extrapolation beyond the grid and both $O(1)$ per call. They are deliberately plain classes rather than closures so that they can be *pickled* and therefore shipped to the worker processes of the parallel zeroth-order solvers.

Each also exposes

```
f = env['gg_upper'].as_callable()      # plain closure, same arithmetic
```

which returns a plain closure evaluating the same expression, for call sites that evaluate the bound many times and do not need to pickle it. **calc_vel_profile_fb2d** applies this automatically to any bound that offers it, so ordinary users need not call it; the instance itself remains what crosses a process boundary.

4.13 Track sampling and reconstruction helpers

These are the operations that every experiment script was re-implementing locally; they live in **utils.py** so that all of them agree.

```
lo_m, hi_m = seg_bounds(reftrack, lo=0.8, hi=1.3)
```

Returns absolute lower and upper segment-length bounds in metres for a track, scaled from the track's own mean node spacing. The bilevel solver optimises segment lengths within such a box; expressing the box relative to the spacing keeps one setting valid across circuits of different size.

```
reftrack_sub, idx = subsample_reftrack(
    reftrack, step,
    mode='arc',          # 'arc' | 'index' | 'seg_unif'
    abscissa=None,
```

```
)
```

Sub-samples a reference track. `'index'` simply keeps every `step`-th row, which is cheap but inherits whatever non-uniformity the source file has. `'arc'` and `'seg_unif'` instead resample at uniform arc length, which matters because short closing segments make the cubic spline curl and corrupt the curvature near the seam. The closing point is always retained.

```
raceline = raceline_from_alpha(reftrack, normvectors, alpha,
                               stepsize_interp=1.0)
```

Builds the interpolated raceline for a lateral-shift vector $\boldsymbol{\alpha}$ in one call, wrapping `create_raceline`. Note the default `stepsize_interp` of 1.0 m: a caller working at a different interpolation step must pass its own value, or it will silently get a different number of raceline points than the rest of its pipeline.

```
(bl_x, bl_y), (br_x, br_y) = track_borders(
    x_mid, y_mid, dir_mid, width_left, width_right)
```

Returns the left and right track borders from a mid-line, its heading and the local half-widths—used for plotting and for corridor audits when the corridor varies along the lap.

```
ggv, ax_max_machines = ggv_tables_from_envelope(env)
```

Collapses an envelope dict back into the two-column `ggv` and `ax_max_machines` tables that the vanilla forward–backward solver and `PointMassOCP` consume, so that one envelope file can feed every engine.

4.14 Centreline projection toolkit

Comparing solutions that were produced by different packages requires putting them on a common abscissa. These helpers do that.

```
v_unwrapped = unwrap_period(v, period)        # undo modulo wrap-
    around
frame       = centreline_frame(x_c, y_c)      # cached arc length +
    tangents
s, n        = project_to_centreline(frame, xy, n_nb=4)
s, n_signed = project_signed(frame, xy, n_nb=4)
y_smooth    = smooth_over(y, x, win_m=60.0)   # moving average over a
    window in m
```

`centreline_frame` precomputes the arc length, tangents and normals of a centreline once; `project_to_centreline` then returns the arc-length station s and lateral offset n of arbitrary points by searching the `n_nb` nearest segments, and `project_signed` signs n with the package's normal convention (normals point to the *right*, so $\alpha = -n$). `unwrap_period` is needed wherever an abscissa has been stored modulo the lap length, which otherwise produces a spurious jump at the seam.

5 Module: `solvers.py`

`solvers.py` contains the high-level optimisation classes and lower-level QP-based geometric solvers.

5.1 `opt_min_curv` – Minimum-Curvature QP

```
alpha_mincurv, curv_error_max = opt_min_curv(
    reftrack, normvectors, A,
    kappa_bound, w_veh,
    print_debug=False, plot_debug=False,
    closed=True,
    n_interp_con=2,
)
```

Solves a QP that minimises the integral of squared curvature of the resulting spline raceline. The optimisation variable is $\boldsymbol{\alpha} \in \mathbb{R}^n$, the vector of lateral shifts applied to each reference-line point along its outward normal:

$$\mathbf{p}_i = \mathbf{r}_i + \alpha_i \, \hat{\mathbf{n}}_i.$$

Mathematical formulation

$$\min_{\boldsymbol{\alpha}} \ \tfrac{1}{2}\boldsymbol{\alpha}^\top H \boldsymbol{\alpha} + \mathbf{f}^\top \boldsymbol{\alpha} \quad \text{subject to} \quad G\boldsymbol{\alpha} \leq \mathbf{h},$$

where H and $\mathbf{f}$ are assembled by `H_f` and the inequalities enforce track-boundary limits. Solved with *quadprog*. See [5] for a detailed derivation.

Parameters

Parameter	Description
`reftrack`	`(n,4)`: x, y, w_{right}, w_{left}
`normvectors`	`(n,2)` unit normals from **`calc_splines`**
`A`	Spline system matrix M from **`calc_splines`**
`kappa_bound`	Maximum curvature [m^{-1}]
`w_veh`	Safety clearance per side [m]
`closed`	`True` for a closed circuit
`n_interp_con`	Interior points constrained per spline segment (default 2; 0 restores the node-only constraints of v≤0.2.2). See Section 4.7

Returns

Return	Description
`alpha_mincurv`	`(n,)` optimal lateral-shift vector [m]
`curv_error_max`	Maximum curvature residual relative to the geometric bound

5.2 `OSP`, `ShortestPath` – Optimal Shortest Path QP Matrices and Feasible Path

```
H, f, G, h = OSP(
    reftrack,          # (n,4): x, y, w_right, w_left
    normvectors,       # (n,2): unit normals
    w_veh,             # vehicle width [m]  (clearance = w_veh/2 per side)
    print_debug=False,
    A=None,            # spline system matrix; enables interior constraints
    closed=True,
```

```
    n_interp_con=2,
)
```

Builds the QP matrices for the *Optimal Shortest Path* problem, which minimises the sum of squared lateral displacements between consecutive raceline points:

$$\min_{\boldsymbol{\alpha}} \sum_{i=0}^{n-1} \|\mathbf{p}_{i+1} - \mathbf{p}_i\|^2, \quad \mathbf{p}_i = \mathbf{r}_i + \alpha_i \, \hat{\mathbf{n}}_i,$$

subject to the box constraint $\alpha_i \in [-(w_{\text{left},i} - w_{\text{veh}}/2),\ w_{\text{right},i} - w_{\text{veh}}/2]$. Note that `w_veh` is the *full* vehicle width; the per-side clearance is `w_veh`/2. More details can be found in [1].

Since v0.2.3, supplying the spline system matrix `A` additionally constrains interior points of each segment (Section 4.7), keeping the interpolated shortest path inside the corridor. When `A` is omitted the function behaves exactly as before and constrains the nodes only.

The convenience wrapper **`ShortestPath`** builds `A` itself and, since v0.2.5, forwards it together with its own `n_interp_con` argument (default 2).

The matrices $(H, \mathbf{f}, G, \mathbf{h})$ are ready for *quadprog*. This function is also used internally by **`Blackbox_raceline`** for the `'shortest'` initialisation mode (passing `w_veh = 2*sfty` so that the per-side clearance matches `sfty`).

```
raceline, alpha_sp, s_splines, vx_profile, ax_profile, kappa_opt,
    t_profile \
    = ShortestPath(
        reftrack, w_veh, stepsize, v_max, plot,
        ggv_import_path, ax_max_machines_import_path,
        vel_solver='fb',
        gg_upper=None, gg_lower=None, gg_range=None,
        dyn_model_exp=1.0)
```

Convenience wrapper: calls **`OSP`** to solve the shortest-path QP, then runs **`create_raceline`** and the full kinematic pipeline, and optionally plots the result. The function returns seven values.

Parameters

Parameter	Description
`reftrack`	`(n,4)`: x, y, w_{right}, w_{left}
`w_veh`	Full vehicle width [m]
`stepsize`	Raceline interpolation step size [m]
`v_max`	maximum possible velocity [m/s]
`plot`	Display plots if `True`
`ggv_import_path`	Path to GGV CSV
`ax_max_machines_import_path`	Path to machine-traction CSV
`vel_solver`	Velocity-profile engine: `'fb'` (default) or `'fb2d'` (FBGA)
`gg_upper, gg_lower, gg_range`	Optional custom g-g bounds for `'fb2d'` (see **`calc_vel_profile_fb2d`**); each `None` entry is rebuilt from the GGV
`dyn_model_exp`	Tyre-model exponent in $[1, 2]$ (1 = diamond, 2 = ellipse) for the velocity profile

Returns

Return	Description
`raceline`	`(M,2)` interpolated raceline $[x, y]$
`alpha_sp`	`(n,)` lateral-shift vector [m]
`s_splines`	`(M,)` cumulative arc-length [m]
`vx_profile`	`(M,)` velocity profile [m/s]
`ax_profile`	`(M,)` longitudinal acceleration [m/s^2]
`kappa_opt`	`(M,)` curvature profile [m^{-1}]
`t_profile`	`(M+1,)` cumulative lap time [s]

5.3 Class `Opt_min_CurvTime`

Since v0.2.4 the constructor also accepts `n_interp_con` (default 2), which is forwarded to the inner minimum-curvature QP so that the *interpolated* raceline is kept inside the corridor and not merely the reference-track nodes (Section 4.7).

`Opt_min_CurvTime` combines spline geometry with a kinematic velocity profile to create a fast surrogate for lap time. A meta-optimiser (CMA-ES or ZORM) searches over the spline *segment-length* vector $\mathbf{s} \in \mathbb{R}^n_{>0}$ to minimise this surrogate. For each candidate $\mathbf{s}$, the objective function **`f_t`** fits splines with those lengths, solves the minimum-curvature QP for $\boldsymbol{\alpha}$, builds the raceline, and returns the predicted lap time.

Constructor

```
obj = Opt_min_CurvTime(
    reftrack,                # (n,4): x, y, w_right, w_left
    center,                  # (n,4): centre-line (same as reftrack for 1-pass runs)
    mu        = 0.05,        # ZO smoothing parameter
    h         = 0.001,       # ZO gradient step size
    kapb      = 0.7,         # max curvature bound [1/m]
    sfty      = 1.0,         # safety clearance per side [m]  (= vehicle half-width)
    t         = 1,           # ZO directions averaged per gradient estimate
    si        = 0.8,         # raceline interpolation step size [m]
    vm        = 22.88,       # maximum velocity [m/s]
    m_veh     = 1000.0,      # vehicle mass [kg]
    drag_coeff = 0.0,        # drag coefficient [kg/m]
    MC        = 1,           # Monte Carlo repetitions for ZO
    min_s     = None,        # explicit lower bound(s): scalar or per-segment array
    max_s     = None,        # explicit upper bound(s): scalar or per-segment array
    sigma     = 0.001,       # initial CMA-ES step size
    iterations_ZO  = 300,    # ZO iteration budget
    iterations_CMA = 30,     # CMA-ES generation budget
    popsize   = 16,          # CMA-ES population size
    ggv_import_path = "maps/ggv.csv",
    ax_max_machines_import_path = "maps/ax_max_machines.csv",
    fw        = 3,           # velocity-profile filter window length
    refine_every    = None,  # path-refinement cadence (see below)
    refine_subsample = 1,    # sub-sampling stride after refinement
    vel_solver = 'fb',       # 'fb' (forward-backward) or 'fb2d' (FBGA)
)
```

```
26      gg_upper = None,          # fb2d: custom drive bound  ax = f(ay, v)
27      gg_lower = None,          # fb2d: custom brake bound  ax = f(ay, v)
28      gg_range = None,          # fb2d: custom lateral bound (
        GgRangeMaxMin or f(v))
29      dyn_model_exp = 1.0,      # tyre-model exponent in [1,2]: 1=diamond
        , 2=ellipse
30      s_scale  = None,          # (lo, hi) -> per-segment bounds [lo*s_i,
        hi*s_i];
31                                # None => DEFAULT_S_SCALE = (0.66, 1.5)
32  )
```

Segment-length bounds (`min_s`, `max_s`, `s_scale`). The outer problem searches over the spline intervals $\mathbf{s} = (s_0, \ldots, s_{n-1})$ inside a box $\mathcal{S}$. Two ways of defining that box are supported.

Per-segment box (default). `s_scale` $= (\ell, u)$ builds

$$s_i \in [\, \ell\, s_i^0,\ u\, s_i^0\,], \qquad i = 0, \ldots, n-1,$$

from each interval's *own* initial length s_i^0. Since v0.2.0 this is the **default**: leaving `s_scale`, `min_s` and `max_s` all unset uses `solvers.DEFAULT_S_SCALE` $= (0.66, 1.5)$.

Global box (legacy). Passing `min_s` and `max_s` as scalars makes every interval share one range, which was the default before v0.2.0 (derived as $s_{\min} = \min_i s_i$ and $s_{\max} = 1.7 \max_i s_i$ when both were left unset). They may also be given as arrays with one entry per segment.

Validation. `s_scale` and `min_s`/`max_s` are mutually exclusive, and `min_s`/`max_s` must be supplied together: giving only one of them previously left the other side silently unbounded, and now raises.

The per-segment form is preferable whenever the reference points are unevenly spaced, which is why it is now the default. With a global box a single short interval lowers $s_{\min}$ for *every* interval (and the longest raises $s_{\max}$), so the outer solver may propose spline parameterisations that lie inside the declared feasible set yet are geometrically degenerate, at which point the inner minimum-curvature QP can become infeasible. The effect is easy to reproduce: on a track whose segment lengths span a factor of twelve, 39 of 40 points drawn uniformly from the global box failed the inner QP, against 0 of 40 for `s_scale` $= (0.6, 1.6)$. When `s_scale` is set, the per-segment bounds are also rebuilt after each in-loop path refinement (Section 5.3), so they stay tied to the current geometry.

Vehicle-dynamics-model exponent (`dyn_model_exp`). Exponent e of the combined tyre limit $(a_x/a_{x,\max})^e + (a_y/a_{y,\max})^e \le 1$ used inside the velocity profile: $e = 1$ is a diamond (linear g-g coupling), $e = 2$ the friction ellipse, and $1 < e < 2$ a rounded diamond in between. It is threaded into every velocity-profile call of this class (`f_t`, `generate_kinProfs`, `Comparison`); the default 1.0 preserves earlier behaviour.

Velocity-profile engine (`vel_solver`). Selects which calculator produces the lap-time surrogate: `'fb'` (default) uses the forward–backward `calc_vel_profile`; `'fb2d'` uses the semi-analytical FBGA `calc_vel_profile_fb2d` (requires *fbga-py*). The choice flows through `f_t`, `generate_kinProfs`, `Comparison`, and the parallel workers. When `vel_solver='fb2d'`, the optional `gg_upper`, `gg_lower` and `gg_range` callables are forwarded to `calc_vel_profile_fb2d` to supply a custom (generic) g-g envelope; any left `None` is rebuilt from `ggv`. They are ignored by the `'fb'` engine.

When `min_s` and `max_s` are both `None` (default) the bounds are set automatically from the Euclidean segment lengths of `reftrack`: $s_{\min} = \min_i \|\mathbf{r}_{i+1} - \mathbf{r}_i\|$ and $s_{\max} = 1.7 \times \max_i \|\mathbf{r}_{i+1} - \mathbf{r}_i\|$.

Path refinement (`refine_every`). When set to a positive integer q, `CurveLenOpt` splits the total iteration budget into blocks of q iterations. After each block (except the last) the current best raceline is extracted, the control points are shifted by the QP-optimal $\boldsymbol{\alpha}$, and the result becomes the new reference track. Track half-widths are updated so the physical boundaries stay in place:

$$w^{\text{new}}_{\text{right},i} = w_{\text{right},i} - \alpha_i, \qquad w^{\text{new}}_{\text{left},i} = w_{\text{left},i} + \alpha_i.$$

Call `reset()` to restore the original track between runs.

Key methods

`f_t(s)` Objective function. Given segment lengths `s`, fits splines, solves the curvature QP, builds the kinematic profile, and returns the predicted lap time.

`CurveLenOpt(solver='ZO', refine_every=None)` Runs the meta-optimisation. `solver` is `'ZO'` (ZORM) or `'CMA'` (CMA-ES). Returns the optimised segment-length vector `ds_opt` and the history of the best lap times. When `refine_every` is set it overrides the instance default for this call only.

`generate_raceline(ds=None, solver='ZO')` Given an optimised segment-length vector `ds` (or runs `CurveLenOpt` if `None`), returns the interpolated raceline coordinates.

`generate_kinProfs(ds=None, solver='ZO', refine_every=None)` Runs the full pipeline and returns: `(s, vx, ax, kappa, t_profile, raceline)` – cumulative distance, velocity, acceleration, curvature, lap-time, and raceline coordinates.

`Comparison(plot='N', output='ZO')` Runs ZO, CMA-ES, minimum-curvature, and centreline solutions, prints a lap-time comparison table, and optionally produces comparison plots. Returns the profiles of the requested `output` case.

`reset()` Restores `self.reftrack` to the original track supplied at construction time, discarding any in-place path refinements.

5.4 Class `ConstrainedCMAES_t`

A self-contained implementation of CMA-ES (Covariance Matrix Adaptation Evolution Strategy) with box-constraint handling via death penalty.

Constructor

```
cma = ConstrainedCMAES_t(
    f,          # callable: objective f(x) -> float
    mean,       # initial mean vector  (n,)
    sigma,      # initial step size (scalar)
    popsize,    # population size lambda
    bounds1,    # upper bounds  (n,)
    bounds2,    # lower bounds  (n,)
)
s_opt = cma.optimize(iterations)
```

Algorithm outline

Each generation the algorithm:

1. Samples λ candidate solutions from $\mathcal{N}(\mathbf{m}, \sigma^2 C)$, clipping to `[bounds2, bounds1]`.
2. Evaluates f at each candidate.
3. Updates the mean $\mathbf{m}$, step-size σ, and covariance C via cumulative step-size adaptation (CSA) and rank-μ update.

See [4] for the full algorithm.

5.5 Class `ZORM`

Zero-Order Random Method (ZORM) is a gradient-free optimiser that estimates the gradient of a μ-smoothed surrogate using random perturbations.

Constructor

```
opt = ZORM(
    func,            # callable: objective f(x) -> float
    x0,              # initial point  (n,)
    num_iterations, # total gradient steps
    mu    = 0.05,    # smoothing / perturbation radius
    h     = 0.001,   # gradient step size
    t     = 1,       # directions averaged per estimate
    grad_type        = 'noth',  # 'noth' (forward) or 'h' (central)
    constraint_type = 'c',     # 'c' (box) or 'notc' (unconstrained)
)
x_opt = opt.optimize(lower_bounds, upper_bounds)
```

Algorithm outline

At each iteration ZORM estimates the gradient of the μ-smoothed objective

$$f_\mu(\mathbf{x}) = \mathbb{E}_{\mathbf{u}\sim\mathcal{N}(0,I)}[f(\mathbf{x}+\mu\mathbf{u})]$$

by a random finite-difference estimator, then projects the gradient-descent step onto the box [lower_bounds, upper_bounds]. With `grad_type='noth'` (forward difference):

$$\hat{g} = \frac{f(\mathbf{x}+\mu\mathbf{u}) - f(\mathbf{x})}{\mu}\,\mathbf{u};$$

with `grad_type='h'` (central difference):

$$\hat{g} = \frac{f(\mathbf{x}+\mu\mathbf{u}) - f(\mathbf{x}-\mu\mathbf{u})}{2\mu}\,\mathbf{u}.$$

See [6] for theoretical background.

5.6 Class `Blackbox_raceline`

Since v0.2.4 the constructor also accepts `n_interp_con` (default 2), which is forwarded to the inner minimum-curvature QP so that the *interpolated* raceline is kept inside the corridor and not merely the reference-track nodes (Section 4.7).

`Blackbox_raceline` optimises the lateral shift vector $\boldsymbol{\alpha} \in \mathbb{R}^N$ *directly* for minimum lap time (or any user-supplied cost function). Unlike **`Opt_min_CurvTime`**, there is no inner QP and no curvature proxy: the true kinematic lap-time oracle is called at every function evaluation.

Motivation

Opt_min_CurvTime uses a two-stage pipeline (ZORM/CMA-ES → QP) in which the outer loop optimises spline segment lengths and the inner QP minimises curvature as a proxy for lap time. The proxy is inexact and couples geometry to the inner QP. **Blackbox_raceline** removes both stages and optimises α directly, trading inner-QP exactness for a higher cost-per-evaluation but a more faithful objective.

Mathematical formulation

$$\min_{\boldsymbol{\alpha}} \ \text{cost}(\boldsymbol{\alpha}) \quad \text{subject to} \quad \alpha_i \in \left[-(w_{\text{left},i} - \delta),\ w_{\text{right},i} - \delta\right] \ \forall i,$$

where $\delta =$ `sfty` is the per-side safety clearance and the box bounds are identical to those used by the internal QP of **Opt_min_CurvTime**. The spline normal vectors $\hat{\mathbf{n}}_i$ are computed once from the Euclidean distances of the original reference track and held fixed throughout optimisation, so the box-projection step is exact at every iteration.

The default cost function evaluates the full kinematic lap time via the selected velocity solver (`vel_solver`; forward–backward by default, or FBGA `Fb2d` when set to `'fb2d'`). Any callable `cost_fn(alpha) -> float` can be substituted. The default projection function is a box function to the feasible alpha values. Any callable `Projection_fn(alpha) -> vector` can be substituted.

Constructor

```
bb = Blackbox_raceline(
    reftrack,                # (N,4): x, y, w_right, w_left  (unclosed)
    ggv,                     # GGV array (K,3) already loaded
    ax_max_machines,         # machine limits (L,2) already loaded
    sfty          = 0.5,     # safety half-width [m]
    v_max         = 22.88,   # maximum velocity [m/s]
    si            = 2.0,     # raceline interpolation step size [m]
    fw            = 3,       # velocity-profile filter window (None = off)
    m_veh         = 1000.0,  # vehicle mass [kg]
    drag_coeff    = 0.75,     # drag coefficient [kg/m]
    dyn_model_exp = 1.0,     # dynamics model exponent in [1, 2]
    cost_fn       = None,    # callable(alpha)->float  or  None
    Projection_fn = None     # callable(alpha)->vector  or  None
    init          = 'random',# initialisation strategy (see below)
    oracle        = 'gaussian', # direction oracle: 'gaussian' or 'sphere'
    mu            = 0.05,    # ZO perturbation radius
    h             = 0.001,   # ZO gradient step size
    t             = 1,       # ZO directions averaged per estimate
    grad_type     = 'noth',  # 'noth' (forward) or 'h' (central)
    iterations    = 300,     # default iteration budget
    kappa_bound   = 0.5,     # curvature bound for 'mincurv' init[1/m]
    seed          = None,    # global random seed
    n_jobs        = 1,       # number of available cpu cores for parallel evaluations (the default cost function)
    vel_solver    = 'fb',    # velocity-profile engine: 'fb' or 'fb2d' (FBGA)
    gg_upper      = None,    # fb2d: custom drive bound  ax = f(ay, v)
    gg_lower      = None,    # fb2d: custom brake bound  ax = f(ay, v)
    gg_range      = None,    # fb2d: custom lateral bound (GgRangeMaxMin or f(v))
)
```

The `vel_solver` argument selects the velocity-profile engine used by the default lap-time oracle: `'fb'` (forward–backward, default) or `'fb2d'` (semi-analytical FBGA, requires *fbga-py*). It is honoured in both the serial and parallel (`n_jobs` > 1) evaluation paths. With `'fb2d'`, the optional `gg_upper`, `gg_lower` and `gg_range` callables define a custom (generic) g-g envelope forwarded to `calc_vel_profile_fb2d`; whichever is `None` is rebuilt from `ggv`. Note that custom callables (Python lambdas / closures, or an `fbga_py GgRangeMaxMin` object) are generally not picklable, so with `n_jobs` > 1 the default oracle automatically falls back from processes to a shared-memory thread pool.

Initialisation modes (`init`).

Mode	Description
`'zero'`	$\alpha_0 = \mathbf{0}$ — start from the reference line
`'random'`	$\alpha_{0,i} \sim \text{Uniform}(\ell_i, u_i)$ — random interior point
`'mincurv'`	Solve the minimum-curvature QP (`H_f`) and use the result as α_0; provides the best geometric warm-start
`'shortest'`	Solve the OSP QP and use the result as α_0; useful on high-speed tracks where path length matters more than apex curvature

Direction oracles (`oracle`).

- `'gaussian'`: $\mathbf{u} \sim \mathcal{N}(0, I_N)$
- `'sphere'`: $\mathbf{u}$ sampled from the unit sphere

Gradient estimators (`grad_type`).

- `'noth'` (forward): $\hat{g} = \frac{f(\alpha+\mu u)-f(\alpha)}{\mu}\, u$
- `'h'` (central): $\hat{g} = \frac{f(\alpha+\mu u)-f(\alpha-\mu u)}{2\mu}\, u$

Method: `find_alpha`

```
best_alpha, history = bb.find_alpha(
    solver      = 'ZO',    # 'ZO' or 'CMA'
    n_iter      = None,    # overrides bb.iterations if set
    # ZO parameters
    step_size   = None,    # overrides bb.h
    mu          = None,    # overrides bb.mu
    t           = None,    # overrides bb.t
    grad_type   = None,    # overrides bb.grad_type
    # CMA parameters
    sigma       = None,    # initial step size (default: mean box width / 6)
    popsize     = None,    # population size (default: 4 + 3*ln(N))
    # Common
    seed        = None,
    verbose     = True,
    print_every = 50,
    feasible_eval = False,
    n_jobs = None
)
```

Minimises the cost over $\boldsymbol{\alpha}$ using either projected ZO gradient descent or CMA-ES. Both solvers maintain and return the *best-ever* alpha seen across all evaluations. If *feasible_eval* is True, all

the points in function evaluations (including perturbations) are projected onto the feasible box before evaluation. This can be useful for cases where the cost function is only defined within the feasible region. By default, the ZO solver evaluates the raw (potentially infeasible) perturbations. Moreover, the black box solver supports parallel function evaluation by utilising different cpu cores through specifying the parameter `n_jobs`.

Returns:

- `best_alpha`: `(N,)` alpha achieving the lowest observed cost
- `history`: `(n_iter+1,)` best-so-far cost at each iteration/generation; `history[0]` is the cost at α_0

After the call, `bb.alpha_opt` and `bb.history` are updated.

Method: `generate_raceline`

```
s, vx, ax, kappa, t_prof, raceline_xy = bb.generate_raceline(alpha=
    None)
```

Builds the full kinematic profiles for a given $\boldsymbol{\alpha}$. If `alpha` is `None`, uses `bb.alpha_opt` (result of the most recent **`find_alpha`** call).

Returns:

Return	Description
`s`	`(M,)` cumulative arc-length [m]
`vx`	`(M,)` velocity profile [m/s] (unclosed)
`ax`	`(M,)` longitudinal acceleration [m/s^2]
`kappa`	`(M,)` curvature [m^{-1}]
`t_prof`	`(M+1,)` cumulative lap time [s]; `t_prof[-1]` = total
`raceline_xy`	`(M,2)` interpolated raceline coordinates

Method: `reset_alpha`

```
bb.reset_alpha(new_init=None)
```

Re-initialises α_0 and clears `alpha_opt` and `history`. Pass `new_init` to switch the initialisation strategy; `None` repeats the current one. Useful for multi-start experiments.

Custom cost functions

Any callable `cost_fn(alpha: np.ndarray) -> float` is accepted. Even the cost function can be focused on other aspects but the lap time. The example below adds a lateral-acceleration comfort penalty to the lap time:

```
W_COMFORT = 0.5    # weight [s / (m/s^2)]

def comfort_cost(alpha):
    s, vx, ax, kappa, t_prof, _ = bb.generate_raceline(alpha)
    laptime = t_prof[-1]
    ay_peak = float(np.max(vx**2 * np.abs(kappa)))  # [m/s^2]
    return laptime + W_COMFORT * ay_peak

bb_comfort = Blackbox_raceline(
    reftrack, ggv, ax_max_machines,
    init='mincurv', cost_fn=comfort_cost)
best_alpha, history = bb_comfort.find_alpha(solver='ZO', n_iter=200)
```

5.7 Class `PointMassOCP` – Point-Mass Minimum-Lap-Time OCP

`PointMassOCP` is an OptiLine front end for the external *MLTS-point-mass* package, which solves the minimum lap time of a point mass in curvilinear coordinates with CasADi/IPOPT. It is the one of the solver in OptiLine that optimises the *path and the speed profile together*: the geometric solvers (Sections 5.1 and 5.2) fix a geometric objective and profile the result afterwards, while the zeroth-order solvers (Sections 5.3 and 5.6) search over lateral shifts with the profiler in the loop.

Formulation. The states are the lateral offset n, the relative heading ξ, the speed V and the two accelerations a_x, a_y; the controls are the commanded accelerations. With $\rho(\zeta)$ the reference-line curvature,

$$\dot{s} = \frac{V\cos\xi}{1-\rho n}, \qquad \frac{\mathrm{d}n}{\mathrm{d}\zeta} = \frac{V\sin\xi}{\dot{s}}, \qquad \frac{\mathrm{d}\xi}{\mathrm{d}\zeta} = \frac{a_y}{V\dot{s}} - \rho, \qquad \frac{\mathrm{d}V}{\mathrm{d}\zeta} = \frac{a_x}{\dot{s}}, \tag{2}$$

and each acceleration follows its command through a first-order lag, $\dot{a} = (a_{\mathrm{ctrl}} - a)/\tau$. The envelope enters as the path constraint $g(a_x, a_y, V) \le 1$ and the corridor as $-w_R + \frac{w_{\mathrm{veh}}}{2} \le n \le w_L - \frac{w_{\mathrm{veh}}}{2}$.

Those lags are the practical difference from a forward–backward velocity profiler such as **`calc_vel_profile`** or **`calc_vel_profile_fb2d`**: those may switch between full braking and full drive between two adjacent samples, whereas here a_x and a_y are continuous states with a finite rate. A lap time obtained from a forward–backward pass is therefore optimistic relative to this solver on the same path and the same envelope.

Usage

```python
from OptiLine.solvers import PointMassOCP

pm = PointMassOCP(
    reftrack,                   # [x, y, w_tr_right, w_tr_left]
    ggv,                        # envelope dict, or [v, ax_max, ay_max] table
    w_veh=1.664, v_max=76.9,
    v_min=5.0, tau_ax=0.03, tau_ay=0.03,
)
pm.solve(mesh=1.0)              # uniform 1 m mesh, or an array of nodes
s, vx, ax, kappa, t_profile, raceline = pm.generate_kinProfs()
print(pm.laptime, pm.envelope_usage())
```

`generate_kinProfs` returns the profiles in the same order as **`Opt_min_CurvTime.generate_kinProfs`** and **`Blackbox_raceline.generate_raceline`**, so the solver drops into existing comparison code. The lateral offset is exposed as **`alpha`** $= -n$ in OptiLine's sign convention (Section 4.10).

Envelope input. Two forms are accepted. A full envelope, given as a dictionary with the keys **`longitudinal_acceleration`** and **`lateral_acceleration`**, is passed straight through. An OptiLine point-mass table $[v, a_{x,\max}, a_{y,\max}]$ is converted by **`ggv_table_to_mlts_samples`**, which walks the boundary of

$$a_{x,\mathrm{avail}}(a_y, v) = a_{x,\max}(v)\left[1 - \left(\frac{|a_y|}{a_{y,\max}(v)}\right)^{e}\right]^{1/e}, \tag{3}$$

with the drive side additionally capped by $a_{x,\max,\mathrm{machines}}(v)$, so that the OCP runs on exactly the same envelope as the rest of the package. That reconstruction can only describe an e-superellipse; prefer the full envelope when one is available.

Reference-track resolution. The solver splines the reference line through the rows of the converted track table and evaluates it at the mesh nodes, so the table must be fine enough for its own curvature. The relevant quantity is $\kappa_{\max} h$, with h the node spacing; the attribute `kappa_h` reports it, and a value above 0.5 raises a `RuntimeWarning`. Measured on the Budapest centre line:

points	$\kappa_{\max} h$	lap, curvilinear	disagreement with (x, y)
220	1.49	32.63 s	0.507 %
439	0.83	32.09 s	0.083 %
876	0.43	31.94 s	0.010 %

Below the warning threshold the lap time read off the returned (x, y) path agrees with the reported one to about 0.01 %. This is a property of the *reference track*, not of the mesh: refining the mesh alone does not remove the disagreement, because the reference-line spline still cuts the corners.

Diagnostics. `envelope_usage` returns the solver's own constraint value $g(a_x, a_y, V)$ summarised as `g_max`, `g_mean` and `frac_on_boundary` (the fraction of nodes with $g > 0.99$). A converged solution has $g_{\max} \leq 1$ to solver tolerance; a value above 1 indicates a solution that does not respect the envelope it was given.

Non-convergence. If IPOPT does not converge, the last iterate is returned and a `RuntimeWarning` is raised; `solver_info` carries `success`, `status`, `iterations` and `solve_time`. A coarser mesh, a different initial guess `x0`, or a larger control-rate weight `w__ax`/`w__ay` usually helps.

5.8 Class `Clothoid_raceline`

Fits a piecewise clothoid (Euler spiral) raceline where curvature varies linearly along each segment.

```
cl = Clothoid_raceline(
    k_0, s, x0, y0, th0,
    nump=10, a_max=5.3, a_min=12, ay_max=12,
    c0=0.00002, c1=0.0015,
    v_max=22.88, v0=0, vf=22.88,
)
x_pts = cl.X_0(a, b, c)    # Fresnel-integral x displacement
y_pts = cl.Y_0(a, b, c)    # Fresnel-integral y displacement
path  = cl.compute_clothoid_path()
```

Fresnel-type integrals are evaluated numerically via *scipy.integrate.quad*. See [3] for the mathematical background.

6 Module: `KinematicProfs.py`

`KinematicProfs.py` computes physically consistent velocity, acceleration, and lap-time profiles along a given curvature profile, using a GGV diagram for the vehicle dynamics limits.

Two velocity-profile engines are available and share the same call conventions: the default forward–backward solver `calc_vel_profile` (Section 6.1) and the semi-analytical FBGA solver `calc_vel_profile_fb2d` (Section 6.2). The thin dispatcher `calc_vel_profile_solver` (Section 6.3) selects between them with a single `vel_solver` keyword, and is the entry point used internally by all solver classes so that the choice propagates package-wide.

6.1 calc_vel_profile

```
vx_profile = calc_vel_profile(
    ggv, ax_max_machines,
    v_max, kappa, el_lengths, closed,
    filt_window = 3,
    drag_coeff  = 0.0,
    m_veh       = 1000.0,
    dyn_model_exp = 1.0,
    mu          = None,
    v_start     = None,
    v_end       = None,
    p_ggv       = None,      # pre-tiled GGV (n,K,3); avoids
    recomputation
)
```

Computes the maximum feasible velocity profile via a forward–backward integration algorithm:

1. **Lateral limit**: $v_i \leq \sqrt{a_{y,\max}(v_i)/|\kappa_i|}$
2. **Forward pass** (acceleration): integrate from a start speed while respecting $a_{x,\max}$ and the GGV ellipse.
3. **Backward pass** (deceleration): integrate backward from an end speed to enforce braking constraints.
4. **Final profile**: element-wise minimum of the three limits.

Closed-circuit periodicity (`closed=True`). For a closed lap the profile must satisfy $v[0] = v[-1]$ (the car enters the next lap at the same speed it left). This is achieved by a **3-lap tripling** approach:

1. The curvature array is tiled three times: $[\kappa, \kappa, \kappa]$.
2. The forward pass is run on the triple and the middle lap is extracted.
3. The backward pass is run on the middle lap result (again tripled) and the middle lap is extracted.

Tripling (rather than doubling) is necessary because the backward pass is implemented by *flipping* and running the forward algorithm. With only two laps, the first element of the flipped array is never written by the forward algorithm, leaving $v[-1]$ unconstrained and causing a large artificial deceleration spike at the lap seam. With three laps the corner at the start of the third copy constrains $v[-1]$ of the middle lap to the physically correct braking distance.

Key parameters

Parameter	Description
`ggv`	`(K,3)` array: $[v, a_{x,\max}, a_{y,\max}]$
`ax_max_machines`	`(L,2)` motor traction table: $[v, a_{x,\text{drive}}]$
`v_max`	Hard speed cap [m/s]
`kappa`	Curvature array [m^{-1}], length n
`el_lengths`	Segment lengths [m], length n
`closed`	`True` for a closed lap (3-lap tripling used)
`filt_window`	Moving-average filter window; `None` disables filtering
`drag_coeff`	Aerodynamic drag [kg/m]; effective deceleration $a_{\text{drag}} = c_{\text{drag}} v^2/m$
`m_veh`	Vehicle mass [kg]
`dyn_model_exp`	GGV ellipse exponent in $[1, 2]$; 1 = diamond, 2 = ellipse
`v_start`	Start velocity [m/s] for the unclosed case
`p_ggv`	Pre-tiled `(n,K,3)` GGV array; skips per-call tiling

6.2 `calc_vel_profile_fb2d`

```
vx_profile = calc_vel_profile_fb2d(
    ax_max_machines, kappa, el_lengths, closed,
    drag_coeff, m_veh,
    ggv          = None,
    v_max        = None,
    dyn_model_exp = 1.0,
    mu           = None,
    v_start      = None,
    v_end        = None,
    filt_window  = None,
    p_ggv        = None,      # accepted and ignored (fb2d builds g-g from ggv)
    gg_upper     = None,      # optional custom drive bound  ax = f(ay, v)
    gg_lower     = None,      # optional custom brake bound  ax = f(ay, v)
    gg_range     = None,      # optional lateral bound (GgRangeMaxMin or f(v))
    ggv_envelope = None,      # envelope dict / JSON path: builds all three above
    n_laps       = 3,         # laps tiled for the closed case (odd)
    return_accel = False,     # also return the longitudinal accel profile
)
```

`ggv_envelope` is the convenient path: pass the dict returned by **`load_ggv_envelope`**, a path to the envelope JSON, or the raw JSON, and `gg_upper`, `gg_lower`, `gg_range` and `v_max` are derived from it. Anything passed explicitly still wins, so a single bound can be overridden while the rest come from the envelope.

Alternative velocity-profile calculator built on the FBGA `Fb2d` solver (*fbga-py*; https://github.com/DRIVEWISE/FBGA). It solves the same minimum-time, fixed-path velocity-profile problem as **`calc_vel_profile`** but integrates the longitudinal dynamics *semi-analytically* (with quadratic drag) rather than by discrete forward–backward stepping, following the method of [3]. It is a

drop-in alternative: the shared parameters carry the same meaning and the returned profile uses the same shape convention (unclosed; length n for a closed input).

Matching the vehicle model. By default the g-g bounds handed to `Fb2d` are constructed so that they reproduce *exactly* the acceleration model of **`calc_ax_poss`** used inside **`calc_vel_profile`**:

$$a_x^{\text{drive}}(a_y, v) = \min\!\big(a_{x,\text{avail}}(a_y, v),\, a_{x,\text{mach}}(v)\big) - \tfrac{c_{\text{drag}}}{m}\, v^2, \qquad a_x^{\text{brake}}(a_y, v) = -a_{x,\text{avail}}(a_y, v) - \tfrac{c_{\text{drag}}}{m}\, v^2,$$

with $a_{x,\text{avail}} = a_{x,\max}(v)\,[1 - (|a_y|/a_{y,\max}(v))^e]^{1/e}$, $e =$ `dyn_model_exp`, and a hard cap at `v_max`. Thus, for the same inputs, both solvers refer to the same car.

Closed-circuit handling. The closed case reuses the same 3-lap tiling trick as **`calc_vel_profile`**: the curvature is tiled `n_laps` times (default 3), `Fb2d` is run over the tiled track, and the middle lap is returned, which is periodic to numerical precision.

Generic acceleration bounds. Passing `gg_upper` / `gg_lower` (callables `f(ay, v) -> ax`) and/or `gg_range` (an `fbga_py GgRangeMaxMin` object, or a callable `f(v) -> ay_max`) overrides the ggv-derived defaults, exposing FBGA's "generic" envelope: velocity dependent, asymmetric in accel/brake, or any measured g-g shape. Whichever of the three is left `None` is rebuilt from `ggv`.

Acceleration output. With `return_accel=True` the function returns the tuple `(vx_profile, ax_profile)`, where `ax_profile` is the longitudinal acceleration produced directly by the solver (same length as `vx_profile`).

Notes

- Requires the optional dependency *fbga-py* (imported lazily; a clear `ImportError` is raised only if the `fb2d` path is used without it installed).
- Only the `ggv` operating mode is supported. Position-dependent inputs (`loc_gg`, or a spatially varying `mu` array) cannot be represented by the single velocity/lateral-dependent g-g functions `Fb2d` expects; a `mu` array is collapsed to its mean.

6.3 `calc_vel_profile_solver`

```
vx_profile = calc_vel_profile_solver(vel_solver='fb', **kwargs)
```

Thin dispatcher that returns a velocity profile from either the vanilla forward–backward solver or the FBGA `Fb2d` alternative, so call sites can offer the choice with a single keyword. The remaining keyword arguments are forwarded unchanged to the selected calculator (identical keyword set to **`calc_vel_profile`**); both branches return the profile with the same shape convention.

`vel_solver`	Calculator
`'fb'` (default)	**`calc_vel_profile`** (forward–backward). Aliases: `'vanilla'`, `'forward_backward'`, `'fb1d'`
`'fb2d'`	**`calc_vel_profile_fb2d`** (FBGA). Alias: `'fbga'`. The `p_ggv` cache, if present, is dropped.

This dispatcher is called internally by **`Opt_min_CurvTime`**, **`Blackbox_raceline`** and **`ShortestPath`**, each of which exposes a `vel_solver` argument (default `'fb'`) together with the optional `gg_upper`, `gg_lower` and `gg_range` bounds forwarded to the `'fb2d'` engine, so existing code is unaffected unless these options are set explicitly.

6.4 calc_ax_profile

```
ax_profile = calc_ax_profile(
    vx_profile, el_lengths, eq_length_output=False)
```

Numerically differentiates the velocity profile to give the longitudinal acceleration $a_x = (v_{i+1}^2 - v_i^2)/(2\,\Delta s_i)$ at each point. Input `vx_profile` should be the closed version (length $n+1$, last element $= v[0]$).

6.5 calc_t_profile

```
t_profile = calc_t_profile(
    vx_profile, el_lengths, t_start=0.0, ax_profile=None)
```

Integrates the velocity profile to give the elapsed time at each sample point. Returns a vector of length $n+1$; the last entry is the total lap time.

6.6 curvature_profile and curvature_profile2

```
kappa = curvature_profile(points)      # via scipy.interpolate.splprep
kappa = curvature_profile2(reftrack)   # spline-derivative method
```

Two alternative functions for computing the curvature of a discrete point sequence. **`curvature_profile`** uses *scipy.interpolate.splprep*; **`curvature_profile2`** uses the raw spline derivatives directly.

6.7 cumulative_distances

```
s = cumulative_distances(el_lengths)
```

Returns the cumulative arc-length from the first point, starting at zero. Equivalent to `np.concatenate([[0], np.cumsum(el_lengths)])`.

7 Module: opt_mintime.py

`opt_mintime` solves a full minimum-lap-time optimal-control problem (OCP) formulated in the curvilinear frame of the track. It uses CasADi to construct the nonlinear programme (NLP) symbolically and IPOPT as the NLP solver. See [2] for the full mathematical formulation.

Function signature

```
(alpha_opt, v_opt, raceline_opt,
 reftrack_interp_out, a_interp_tmp,
 normvec_out, s_opt, laptime) = opt_mintime(
    reftrack,           # (n,4): x, y, w_right, w_left
    coeffs_x,           # (n,4) spline x-coefficients
    coeffs_y,           # (n,4) spline y-coefficients
    normvectors,        # (n,2) unit normals
    pars,               # dict of vehicle and solver parameters
    print_debug = False,
    plot_       = False,
)
```

Vehicle and tyre parameters are supplied via the `pars` dictionary, which is typically loaded from `params/racecar.ini`. Key sub-dicts include `vehicle_params_mintime`, `tire_params_mintime`, and `optim_opts`.

8 Module: `map_builder.py`

`map_builder.py` generates closed-circuit race tracks procedurally and saves them in the same three-file structure used by the 25 real-circuit maps shipped with OptiLine-Py:

- `<name>_centerline.csv` – the track centreline as rows `x_m, y_m, w_tr_right_m, w_tr_left_m`, with the first point at $(0, 0)$ and consecutive points spaced ≈ 0.4 m apart;
- `<name>_map.png` – a 2000×2000 pixel greyscale occupancy-grid image (ROS convention: 255 = free, 0 = wall, 205 = unknown);
- `<name>_map.yaml` – ROS-style metadata (image filename, resolution in m/px, world-frame origin).

An additional **25 synthetically generated maps** are provided in `maps/Zoo_Maps/example_1/` through `example_25/`. An **unlimited number of additional maps** can be generated on demand, enabling the construction of arbitrarily large datasets for machine-learning and data-driven racing-line optimisation research.

8.1 Track generation pipeline

Generating a single track involves four sequential steps:

1. **Irregular polygon backbone.** N vertices are distributed at random angles around a scaled ellipse, with $\pm 38\,\%$ radial jitter and a per-vertex probability (`conc`) of being pushed further inward to 35–65 % of the base radius, producing concavities that mimic the inward peninsulas typical of street circuits.
2. **3-point corner stiffening.** For every polygon vertex v, three control points are inserted: a point `stiff` m before v along the incoming edge (*enter*), v itself, and a point `stiff` m after v along the outgoing edge (*leave*). This forces the periodic cubic spline to make a sharp turn near v while the gaps between consecutive leave–enter pairs remain near-perfectly straight. The stiffening distance ranges from 3.5 m (street circuits) to 6.5 m (tilodrome).
3. **Feature injection.** In the straight sections between consecutive vertices, two types of feature can be inserted:
   - *Hairpin* – two apex control points pushed inward toward the track centroid by 50–75 % of the local radius.
   - *Chicane* – two laterally offset control points forming an S-bend with a random left/right sense.
4. **Periodic cubic spline + validation.** A `scipy.interpolate.CubicSpline` with `bc_type ='periodic'` is fitted through all control points using cumulative chord-length parametrisation, guaranteeing exact closure. The dense sample is lightly smoothed ($\sigma \approx 0.5$ m Gaussian, `mode='wrap'`) to suppress numerical oscillations. The centreline is resampled at 0.40 m spacing and scaled to the desired total length (default 450 m). A fast grid-hash proximity check rejects self-intersecting candidates; up to 30 retries are performed before falling back to a plain oval.

8.2 Track archetypes

Five named archetypes control the backbone geometry and feature density. When no style is specified, one is chosen uniformly at random.

Table 1: Track-style configuration parameters.

Style	Vertices	Aspect	Stiff (m)	Character
`oval_complex`	4–6	2.2–3.5	5.5	Elongated, few sharp corners
`circuit`	5–8	1.4–2.2	5.0	Balanced hairpins & chicanes
`circuit_complex`	6–9	1.1–1.8	4.5	Compact, dense corner count
`street_circuit`	6–10	1.0–1.5	3.5	Near-square, right-angle bends
`tilodrome`	5–7	1.5–2.6	6.5	Wide sweeping corners

8.3 Class `MapBuilder`

`MapBuilder` is the primary object-oriented interface. It captures default settings (seed, output directory, width mode, target length) so that repeated calls do not require re-specifying them.

Constructor

```
from OptiLine.map_builder import MapBuilder

mb = MapBuilder(
    seed           = 42,      # master random seed
    maps_dir       = None,    # output directory (auto-detected if None)
    variable_width = False,   # smooth Fourier width profile if True
    target_length  = 450.0,   # desired track length [m]
    half_width     = 1.10,    # uniform half-width [m] (variable_width=False)
    w_min          = 0.8,     # min half-width [m] (variable_width=True)
    w_max          = 3.0,     # max half-width [m] (variable_width=True)
)
```

When `maps_dir` is `None` the constructor auto-detects the project root and uses `maps/Zoo_Maps/` if it exists.

Method: `generate_track`

```
track = mb.generate_track(style=None)
# Returns dict with keys: 'xy', 'w_right', 'w_left', 'style'
xy    = track['xy']       # (N, 2)  centerline points [m]
w_r   = track['w_right'] # (N,)    right half-widths [m]
w_l   = track['w_left']  # (N,)    left half-widths [m]
style = track['style']   # str, one of TRACK_STYLES
```

Generates a single random track without any file I/O. Pass `style` to fix the archetype; `None` selects uniformly at random.

Method: `generate`

```
folders = mb.generate(n=10, start_index=1,
                      maps_dir=None, verbose=True)
```

Generates `n` tracks and writes the three files for each to disk. Returns a list of absolute folder paths. `start_index` controls the numbering: `start_index=26` produces `example_26`, `example_27`, ...

Method: `reset`

```
mb.reset(seed=None)    # None reuses the original seed
```

Resets the internal RNG so the same track sequence can be reproduced exactly.

Static method: `render`

```
img = MapBuilder.render(xy, w_right, w_left, resolution, origin)
img.save("track.png")
```

Renders an occupancy-grid PNG from pre-computed geometry without writing the centreline CSV or YAML files.

8.4 Functional API

The module-level functions are available for scripting without instantiating **`MapBuilder`**:

```
from OptiLine.map_builder import generate_random_track,
    build_examples

# Single track (no file I/O)
rng   = np.random.default_rng(42)
track = generate_random_track(rng, target_length=500.0, style='
    circuit')

# Batch generation
build_examples(n=25, maps_dir='maps/Zoo_Maps', seed=7)
```

8.5 Command-line usage

```
# 5 maps, reproducible seed, saved to maps/Zoo_Maps/
python tests/Map_builder_batch_gen.py --n 5 --seed 42

# 1000 maps starting at index 26 (extends the Zoo_Maps dataset)
python tests/Map_builder_batch_gen.py --n 1000 --seed 0 --start-index
    26

# Variable track width (0.8 -- 3.0 m, smooth Fourier profile)
python tests/Map_builder_batch_gen.py --n 10 --variable-width

# Custom output directory
python tests/Map_builder_batch_gen.py --n 20 --maps-dir /path/to/
    output
```

9 Usage Examples

The following scripts, located in the `tests/` directory, provide complete end-to-end demonstrations of the package.

9.1 `General_test.py` – Geometric, Proxy, and Blackbox Optimisation

Run from the `tests/` directory:

```
cd tests && python General_test.py
```

Stage 1 – Track import and preprocessing

Loads the Catalunya centreline CSV, sub-samples to reduce computation time, and imports the GGV and `ax_max_machines` tables.

```
MAP_PATH = "maps/Catalunya/Catalunya_centerline.csv"
csv_data = np.loadtxt(MAP_PATH, comments='#', delimiter=',')
reftrack_full = csv_data[:, 0:4]          # [x, y, w_right, w_left]
reftrack = subsample_track(reftrack_full, step=4)
ggv, ax_max_machines = import_veh_dyn_info(GGV_PATH, AX_MAX_PATH)
```

Stage 2 – Spline computations

Fits C2-continuous cubic splines through the reference points, recovers segment lengths, interpolates at a uniform 2 m step, and computes analytical and numerical heading and curvature profiles.

```
coeffs_x, coeffs_y, M, normvec_norm = calc_splines(
    path=np.vstack((reftrack[:, :2], reftrack[0, :2])),
    el_lengths=ds_0, use_dist_scaling=True)
spline_lengths = calc_spline_lengths(coeffs_x, coeffs_y)
path_interp, spline_inds, t_vals, s_pts = interp_splines(
    coeffs_x, coeffs_y, spline_lengths, stepsize_approx=2.0)
psi_an, kappa_an = calc_head_curv_an(coeffs_x, coeffs_y,
                                     spline_inds, t_vals)
```

Stage 3 – Minimum-curvature raceline

Solves the curvature-minimisation QP, then reconstructs the raceline and computes velocity and lap-time profiles.

```
alpha_mincurv, curv_error = solvers.opt_min_curv(
    reftrack=reftrack, normvectors=normvec_norm, A=M,
    kappa_bound=0.5, w_veh=1.0)
raceline_mc, *_ = create_raceline(
    reftrack[:, :2], normvec_norm, alpha_mincurv, stepsize_interp
    =2.0)
vx_mc = calc_vel_profile(ggv, ax_max_machines, v_max=22.88,
                         kappa=kappa_mc, el_lengths=el_mc, closed=
    True,
                         drag_coeff=0.75, m_veh=1000)
t_mc  = calc_t_profile(vx_mc, ax_mc, el_mc)
```

Stage 4 – Shortest-path raceline

Solves the shortest-path QP and returns the full raceline and kinematic profiles.

```
(raceline_sp, alpha_sp, s_sp,
 vx_sp, ax_sp, kappa_sp, t_sp) = solvers.ShortestPath(
    reftrack=reftrack, w_veh=1.0, stepsize=2.0, plot=False,
    ggv_import_path=GGV_PATH,
    ax_max_machines_import_path=AX_MAX_PATH)
```

Stage 5 – Kinematic profiles

Demonstrates **`KinematicProfs`** in isolation using the minimum-curvature raceline from Stage 3: plots velocity, longitudinal acceleration, and curvature against arc length.

Stage 6 – Full optimisation comparison (ZO vs CMA-ES)

Instantiates `Opt_min_CurvTime` and benchmarks ZORM, CMA-ES, minimum-curvature, and centre-line solutions.

```
optct = solvers.Opt_min_CurvTime(
    reftrack=reftrack, center=reftrack, mu=0.01, h=0.001,
    kapb=0.5, sfty=1.0, si=2, vm=22.88, m_veh=1000,
    drag_coeff=0.75, MC=1, sigma=0.005,
    iterations_ZO=100, iterations_CMA=10, popsize=16,
    ggv_import_path=GGV_PATH,
    ax_max_machines_import_path=AX_MAX_PATH, fw=3)

raceline_zo, ds_zo = optct.generate_raceline(solver='ZO')
s_zo, vx_zo, ax_zo, kappa_zo, t_zo, rl_zo = \
    optct.generate_kinProfs(ds=ds_zo)
s_opt, vx_opt, ax_opt, kappa_opt, t_opt, rl_opt = \
    optct.Comparison(plot='Y', output='ZO')
```

Stage 7 – In-loop path refinement

Demonstrates the `refine_every` feature of `Opt_min_CurvTime`. The total iteration budget is split into blocks of q iterations; after each block (except the last) the current best raceline is used to rebuild the reference track, and optimisation continues on the geometrically tighter track. Three variants are compared head-to-head with the same total budget and the same optimizer instance; a call to `reset()` restores the original track between variants.

```
REFINE_Q_ZO  = 33   # ZO  cadence: 100 iters -> 3 blocks, 2
    refinements
REFINE_Q_CMA =  3   # CMA cadence:  10 iters -> 3 blocks, 2
    refinements

optct_ref = solvers.Opt_min_CurvTime(
    reftrack=reftrack, center=reftrack,
    mu=0.01, h=0.001, kapb=0.5, sfty=1.0, si=2,
    vm=22.88, m_veh=1000, drag_coeff=0.75, MC=1,
    sigma=0.005, iterations_ZO=100, iterations_CMA=10,
    popsize=16, fw=3,
    ggv_import_path=GGV_PATH,
    ax_max_machines_import_path=AX_MAX_PATH,
    refine_every=REFINE_Q_ZO,   # instance default for ZO calls
    refine_subsample=1,         # keep all refined control points
)

# A: ZO without refinement  (override refine_every=0)
s_a, vx_a, ax_a, kappa_a, t_a, rl_a = optct_ref.generate_kinProfs(
    solver='ZO', refine_every=0)
optct_ref.reset()

# B: ZO with in-loop refinement  (uses instance default refine_every)
s_b, vx_b, ax_b, kappa_b, t_b, rl_b = optct_ref.generate_kinProfs(
    solver='ZO')
optct_ref.reset()

# C: CMA-ES with in-loop refinement  (cadence overridden per-call)
s_c, vx_c, ax_c, kappa_c, t_c, rl_c = optct_ref.generate_kinProfs(
    solver='CMA', refine_every=REFINE_Q_CMA)
optct_ref.reset()
```

The output includes a results table and a two-panel figure: the left panel overlays the three racelines on the track boundaries; the right panel compares the velocity profiles.

Variant	Lap time [s]
A. ZO – no refinement (100 iters)	t_A
B. ZO – refine every 33 iters (100 iters)	t_B
C. CMA-ES – refine every 3 iters (10 iters)	t_C

Note on refinement behaviour. After each refinement `self.reftrack` is replaced by the current best raceline and the segment-length bounds (`min_s`, `max_s`) are recomputed from the new control-point spacing. With small per-block budgets the ZO optimiser does not fully converge before the next refinement resets the search landscape. Call **`reset()`** at any time to restore the original track.

Stage 8 – Blackbox direct-alpha optimisation

Demonstrates **`Blackbox_raceline`** with all four initialisation strategies and both solvers, then shows a custom comfort-aware cost function. The stage produces convergence-history plots, per-init improvement comparisons (raceline maps, velocity profiles, Δv_x profiles), and alpha-shift profiles.

```
BB_SFTY = 0.2;  BB_ITERS_ZO = 450;  BB_ITERS_CMA = 30

# --- 8a: initial cost comparison ---
for mode in ['random', 'zero', 'mincurv', 'shortest']:
    bb = solvers.Blackbox_raceline(
        reftrack=reftrack, ggv=ggv, ax_max_machines=ax_max_machines,
        sfty=BB_SFTY, v_max=22.88, si=2.0, init=mode)
    t0 = bb._eval_cost(bb.alpha_0)
    print(f"  {mode}: laptime0 = {t0:.4f} s")

# --- 8b: ZO on all inits ---
for mode in ['random', 'zero', 'mincurv', 'shortest']:
    bb_zo = solvers.Blackbox_raceline(
        reftrack=reftrack, ggv=ggv, ax_max_machines=ax_max_machines,
        sfty=BB_SFTY, v_max=22.88, si=2.0, init=mode,
        mu=0.001, h=0.0001, t=2)
    best_a, hist = bb_zo.find_alpha(
        solver='ZO', n_iter=BB_ITERS_ZO, verbose=True)

# --- 8c: CMA-ES on mincurv and shortest ---
for mode in ['mincurv', 'shortest']:
    bb_cma = solvers.Blackbox_raceline(..., init=mode)
    best_a, hist = bb_cma.find_alpha(
        solver='CMA', n_iter=BB_ITERS_CMA,
        sigma=0.1, popsize=20)

# --- 8d: custom cost function ---
def comfort_cost(alpha):
    s, vx, ax, kappa, t_prof, _ = _bb_ref.generate_raceline(alpha)
    return t_prof[-1] + 0.5 * np.max(vx**2 * np.abs(kappa))

bb_custom = solvers.Blackbox_raceline(
    ..., init='shortest', cost_fn=comfort_cost)
best_custom, hist_custom = bb_custom.find_alpha(
    solver='ZO', n_iter=BB_ITERS_ZO)

```

```
# --- 8e/8f: profiles and plots ---
s, vx, ax, kappa, t_prof, raceline = bb_zo.generate_raceline()
print(f"ZO laptime: {t_prof[-1]:.3f} s")
```

Stage 10 – FBGA `fb2d` velocity profile (alternative `vel_solver`)

Computes the velocity profile on the same minimum-curvature raceline with both engines — the vanilla forward–backward solver and the FBGA `Fb2d` solver — using identical parameters, and reports the lap times with the same **`calc_t_profile`** for a consistent comparison. The stage also exercises the acceleration output, the generic custom-bounds path, the dispatcher aliases, and the end-to-end `vel_solver` option on **`ShortestPath`**. If *fbga-py* is not installed the stage prints a `SKIP` message instead of failing.

```
from OptiLine.KinematicProfs import (
    calc_vel_profile_fb2d, calc_vel_profile_solver)

common = dict(ggv=ggv, ax_max_machines=ax_max_machines, v_max=22.88,
              kappa=kappa_mc, el_lengths=el_mc, closed=True,
              filt_window=3, dyn_model_exp=1.0,
              drag_coeff=0.75, m_veh=1000.0)

# both engines through the dispatcher
vx_fb   = calc_vel_profile_solver(vel_solver='fb',   **common)
vx_fb2d = calc_vel_profile_solver(vel_solver='fb2d', **common)

# fb2d also returns the longitudinal acceleration profile
vx_fb2d, ax_fb2d = calc_vel_profile_fb2d(return_accel=True, **common)

# consistent lap times via calc_t_profile for both engines
def laptime(vx, el):
    vx_cl = np.append(vx, vx[0])
    ax = calc_ax_profile(vx_cl, el, eq_length_output=False)
    return calc_t_profile(vx, el, ax_profile=ax)[-1]
```

The `vel_solver` switch is also accepted directly by the solver classes/functions, e.g. `ShortestPath(..., vel_solver='fb2d')`, `Opt_min_CurvTime(..., vel_solver='fb2d')`, and `Blackbox_raceline(..., vel_solver='fb2d')`.

Stage 9 – Parallel speedup (`n_jobs` > 1)

Runs **`Blackbox_raceline`** with `n_jobs`$= 1$ and `n_jobs`$= 12$ under a fixed seed, for both the ZO and the CMA-ES solver, and reports the wall-clock ratio. The seed is fixed deliberately: the serial and parallel runs must return the *same* lap time, so any difference would indicate a worker that did not receive identical state. Note that the g-g bound objects are shipped to the workers by pickling, which is why **`Bilinear`** and **`Linear1D`** are plain classes.

Stage 11 – Per-segment spline-length bounds

Exercises **`seg_bounds`** and the per-segment length box of **`Opt_min_CurvTime`**: the bilevel solver optimises the spline segment lengths within bounds derived from the track's own node spacing, which keeps one setting valid across circuits of different size.

Stage 12 – Interior-point corridor constraints

Sweeps `n_interp_con` and measures the interior overshoot of the *interpolated* raceline, confirming that the extra constraint rows of **`interp_corridor_rows`** (Section 4.7) keep the spline inside

the corridor between the nodes, not only at them.

Stage 13 – Point-mass minimum-lap-time OCP

Solves `PointMassOCP` on the same track and checks the chain end to end: that `reftrack_to_mlts_track` produces a track table whose abscissa really is the arc length of the curve its own coordinates describe, that the solved raceline respects the thinned corridor, that the solution stays inside $g(a_x, a_y, V) \leq 1$, and that the curvilinear lap time agrees with the time implied by the returned (x, y) path. That last check is the one an inconsistent track file breaks.

Stage 14 – Shared utilities

Equivalence checks for the helpers of Section 4: each is asserted against the longhand it replaced, so that a change to one of them cannot silently move a lap time. Covered are `spline_extraction_ops` against the explicit extraction products, `interp_corridor_rows` with and without a supplied `A_inv`, the sparse spline solve against a dense one, both reconstructions of `load_ggv_envelope` (including that the bound objects survive pickling and that `as_callable()` agrees with `__call__`), `calc_vel_profile_fb2d` driven by `ggv_envelope` against the same bounds passed individually, the sampling and reconstruction helpers, the projection toolkit (which pins the $\alpha = -n$ sign convention), and that the blackbox cost oracle rejects a non-finite lap time instead of propagating `NaN` into its gradient estimate.

9.2 `General_test_optmintime.py` – Minimum Lap-Time Optimal Control

```
cd tests && python General_test_optmintime.py
```

Stage 1 – Track import

Loads and sub-samples the centreline identically to `General_test.py`.

Stage 2 – Configuration loading

Parses `params/racecar.ini` with `configparser` to populate vehicle and solver parameter dictionaries, and loads the GGV table.

```
parser = configparser.ConfigParser()
parser.read("params/racecar.ini")
pars["vehicle_params_mintime"] = json.loads(
    parser.get('OPTIMIZATION_OPTIONS', 'vehicle_params_mintime'))
if pars["optim_opts"]["ax_pos_safe"] is None:
    pars["optim_opts"]["ax_pos_safe"] = np.amin(ggv[:, 1])
```

Stage 3 – Track preprocessing

Calls `utils.prep_track` to smooth and re-sample the centreline at the collocation stepsize and to compute spline coefficients and unit normals.

```
reftrack_interp, normvec_normalized_interp, a_interp, \
    coeffs_x_interp, coeffs_y_interp = utils.prep_track(
        reftrack_imp=reftrack,
        reg_smooth_opts=pars["reg_smooth_opts"],
        stepsize_opts=pars["stepsize_opts"],
        debug=True)
```

Stage 4 – Minimum lap-time optimisation (pass 1)

Solves the full OCP via Gauss–Legendre collocation and IPOPT.

```
(alpha_opt, v_opt, raceline_opt, reftrack_interp_out,
 a_interp_tmp, normvec_out, s_opt, laptime) = opt_mintime.opt_mintime
    (
     reftrack=reftrack_interp,
     coeffs_x=coeffs_x_interp, coeffs_y=coeffs_y_interp,
     normvectors=normvec_normalized_interp,
     pars=pars, print_debug=True, plot_=True)
```

Stage 5 – Re-optimisation pass

Widens the optimisation corridor and re-solves the NLP using the pass-1 track geometry as the new reference.

```
pars_reopt = copy.deepcopy(pars)
pars_reopt["optim_opts"]["width_opt"] = w_veh_widened
alpha_reopt, v_reopt, *_, laptime_reopt = opt_mintime.opt_mintime(
    reftrack=reftrack_interp_out, normvectors=normvec_out,
    pars=pars_reopt, ...)
print(f"Pass 1: {laptime:.2f} s   Pass 2: {laptime_reopt:.2f} s")
```

9.3 `map_building_test.py` – MapBuilder Demonstrations

```
poetry run python tests/map_building_test.py
```

Stage 1 – Per-style gallery

Generates one track per archetype without file I/O and displays all five side by side, useful for visually inspecting the style parameters.

```
mb = MapBuilder(seed=7, target_length=450.0)
for style in TRACK_STYLES:
    track = mb.generate_track(style=style)
    # track: {'xy': (N,2), 'w_right': (N,), 'w_left': (N,), 'style':
    str}
```

Stage 2 – Batch generation to disk

Generates 8 maps into a temporary directory and verifies that all three output files (`_centerline.csv`, `_map.png`, `_map.yaml`) are present for each example.

```
with tempfile.TemporaryDirectory() as tmp:
    mb = MapBuilder(seed=8, variable_width=False)
    folders = mb.generate(n=8, start_index=1,
                          maps_dir=tmp, verbose=True)
```

Stage 3 – Variable-width tracks

Generates 3 tracks with `variable_width=True` and plots both the track layout and the smooth Fourier-based half-width profile for each.

```
mb = MapBuilder(seed=9, variable_width=True, w_min=0.6, w_max=2.5)
track = mb.generate_track()    # widths vary along the track
```

Stage 4 – Reproducibility

Verifies that two **MapBuilder** instances with the same seed produce identical tracks, that **reset()** restores the original sequence, and that a different seed produces genuinely different tracks.

```
mb1 = MapBuilder(seed=42);  mb2 = MapBuilder(seed=42)
tracks1 = [mb1.generate_track() for _ in range(5)]
tracks2 = [mb2.generate_track() for _ in range(5)]
assert all(np.allclose(t1['xy'], t2['xy'])
           for t1, t2 in zip(tracks1, tracks2))
mb1.reset()
tracks1b = [mb1.generate_track() for _ in range(5)]
assert all(np.allclose(t1['xy'], t2['xy'])
           for t1, t2 in zip(tracks1, tracks1b))
```

Stage 5 – Statistics

Generates 50 tracks in-memory (no file I/O) and plots the track-length distribution and style-frequency histogram.

9.4 Map_builder_batch_gen.py – Command-Line Batch Generation

A lightweight CLI script that delegates entirely to `OptiLine-Py.map_builder`. Run from the project root:

```
# 25 maps with a fixed seed (default behaviour)
poetry run python tests/Map_builder_batch_gen.py --n 25 --seed 7

# Extend an existing dataset starting at index 26
poetry run python tests/Map_builder_batch_gen.py --n 1000 --seed 0 \
    --start-index 26

# Variable track widths (0.8 -- 3.0 m, Fourier profile)
poetry run python tests/Map_builder_batch_gen.py --n 10 --variable-
    width

# Write to a custom directory
poetry run python tests/Map_builder_batch_gen.py --n 20 \
    --maps-dir /path/to/output
```

Argument	Description
`-n`	Number of maps to generate (default: 25)
`-seed`	Random seed for reproducibility (default: 7)
`-start-index`	Index of the first example (default: 1 → `example_1`)
`-variable-width`	Enable smooth Fourier width profile instead of uniform
`-maps-dir`	Output directory; auto-detected from project layout if omitted

Because the seed is deterministic, running the script twice with the same arguments produces byte-for-byte identical outputs. The dataset can be extended incrementally: passing `-start-index 26` appends new examples without recomputing the existing ones.

10 GGV Diagram Format

The GGV file must be comma-separated with **no header row**:

Column 1 v [m/s]	Column 2 $a_{x,\max}$ [m/s^2]	Column 3 $a_{y,\max}$ [m/s^2]
0.0	10.0	12.0
10.0	10.0	11.5
20.0	9.5	10.0
⋮	⋮	⋮

The `ax_max_machines` file has two columns: $[v,\ a_{x,\text{drive}}]$. Both files must be monotone in velocity and cover the full range up to `v_max`.

11 Track Map Format

Track CSV files must have **four columns** and **no header row**:

Column 1 x [m]	Column 2 y [m]	Column 3 w_{right} [m]	Column 4 w_{left} [m]
0.00	0.00	5.0	5.0
5.23	0.12	5.0	5.0
⋮	⋮	⋮	⋮

Sign convention: w_{right} is the distance from the reference line to the right boundary (measured along the outward normal); w_{left} is the distance to the left boundary. Both are positive.

The track is assumed **closed**: the last point connects back to the first. Do *not* repeat the first point at the end of the file.

12 Dependencies and Licences

Library	**Licence**	**URL**
NumPy	BSD-3-Clause	https://numpy.org
SciPy	BSD-3-Clause	https://scipy.org
Matplotlib	PSF/Matplotlib	https://matplotlib.org
Pillow	MIT-CMU	https://python-pillow.org
quadprog	GPL-2.0	https://github.com/quadprog/quadprog
CasADi	LGPL-3.0	https://web.casadi.org
IPOPT	EPL-2.0	https://coin-or.github.io/Ipopt
fbga-py	MIT	https://github.com/DRIVEWISE/FBGA

12 References

[1] Francesco Braghin, Federico Cheli, Stefano Melzi, and Edoardo Sabbioni. Race driver model. *Computers & Structures*, 86(13-14):1503–1516, 2008.

[2] Fabian Christ, Alexander Wischnewski, Alexander Heilmeier, and Boris Lohmann. Time-optimal trajectory planning for a race car considering variable tyre-road friction coefficients. *Vehicle system dynamics*, 59(4):588–612, 2021.

[3] Marco Frego, Enrico Bertolazzi, Francesco Biral, Daniele Fontanelli, and Luigi Palopoli. Semi-analytical minimum time solutions with velocity constraints for trajectory following of vehicles. *Automatica*, 86:18–28, 2017.

[4] Nikolaus Hansen. The cma evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2016.

[5] Alexander Heilmeier, Alexander Wischnewski, Leonhard Hermansdorfer, Johannes Betz, Markus Lienkamp, and Boris Lohmann. Minimum curvature trajectory planning and control for an autonomous race car. *Vehicle System Dynamics*, 2020.

[6] Yurii Nesterov and Vladimir Spokoiny. Random gradient-free minimization of convex functions. 17(2):527–566.